



AI-голосовой агент для обработки входящих звонков перевозчиков

Исполнитель: ASRP · **Роль:** ASRP полностью владела фичей голосового ИИ (проектирование, реализация, дорожная карта) · **Тип сотрудничества:** независимый подрядчик · **Домен:** облачная телефония для грузовой логистики

Обезличенный кейс-стади. Идентичность клиента, проприетарный исходный код, внутренние идентификаторы и чувствительные с точки зрения безопасности детали опущены. Сторонние платформы названы только там, где это публичные продукты.

1. Резюме

Клиент — это облачная платформа телефонии для грузовой логистики, работающая между **перевозчиками** (транспортными компаниями, перевозящими грузы) и **брокерами** (которые организуют перевозку). При каждом входящем звонке от перевозчика платформа уже обогащает звонок бизнес-контекстом: сверяет номер звонящего с базами данных, проверяет, привязан ли он к известному перевозчику, и прогоняет звонок через классическое IVR-меню (“введите ваш MC-номер”, “введите номер груза”), прежде чем звонок доходит до человека.

ASRP спроектировала и построила **голосового ИИ-агента**, который апгрейдит это IVR. Когда номер груза, названный звонящим, не удаётся сопоставить, звонок передаётся диалоговому агенту, который идентифицирует отгрузку по голосу — по референс-номеру либо по городам отправления/назначения и датам — а затем сообщает структурированный результат, чтобы бэкэнд мог направить квалифицированный звонок в очередь на живого оператора. Агент **диалоговый для звонящего, но строго детерминированный внутри**: он никогда не выдумывает данные о грузе, никогда не решает судьбу звонка самостоятельно и корректно откатывается к классическому IVR при любом сбое.

Система интегрирует речевую нативную ИИ-платформу **Ultravox**¹ в уже существующий стек телефонии платформы на базе **Jambonz**² + **Bandwidth**³ по четырём независимым каналам коммуникации, с транскрипцией в реальном времени, событийно-ориентированной маршрутизацией и выделенным контуром для тестирования и самообучения.

2. Бизнес-контекст и проблема

Ценность платформы — превращать обычный телефонный звонок в **обогащённое, верифицированное бизнес-событие**: кто звонит, от какой транспортной компании, по какому грузу и зачем. Один только номер звонящего мало что доказывает, поэтому платформа связывает номер с

идентичностью MC/DOT, статусом полномочий перевозчика, связанными перевозчиками, историей звонков и статусом комплаенса — ещё до того, как это придётся выяснять человеку.

Слабым местом была **первая стадия звонка**. Базовый сценарий: перевозчик звонит на номер брокера → вводит MC-номер и номер груза в IVR → его направляют к человеку. К моменту, когда человек берёт трубку, он знает **КТО** звонит (по MC) и **ПО КАКОМУ** грузу (по введённому номеру) — но **не ЗАЧЕМ**, и нет никакой преемственности между звонками. На практике это приводило к следующему:

- **Отсутствие «зачем» при передаче** — человек берёт трубку, не зная причины звонка.
- **Отсутствие преемственности** — на просьбу «переключите меня обратно на человека, с которым я только что говорил» нет ответа; звонящие перезванивают снова и снова, чтобы что-то уточнить или узнать статус.
- **Тупики** — жёсткое меню не может обработать перевозчика, у которого нет точного номера груза, или естественный запрос вроде «я водитель, который забирает груз в Чикаго завтра».

Задача: заменить жёсткую первую стадию агентом, который устанавливает недостающие факты **до** подключения человека и переносит этот контекст через передачу звонка.

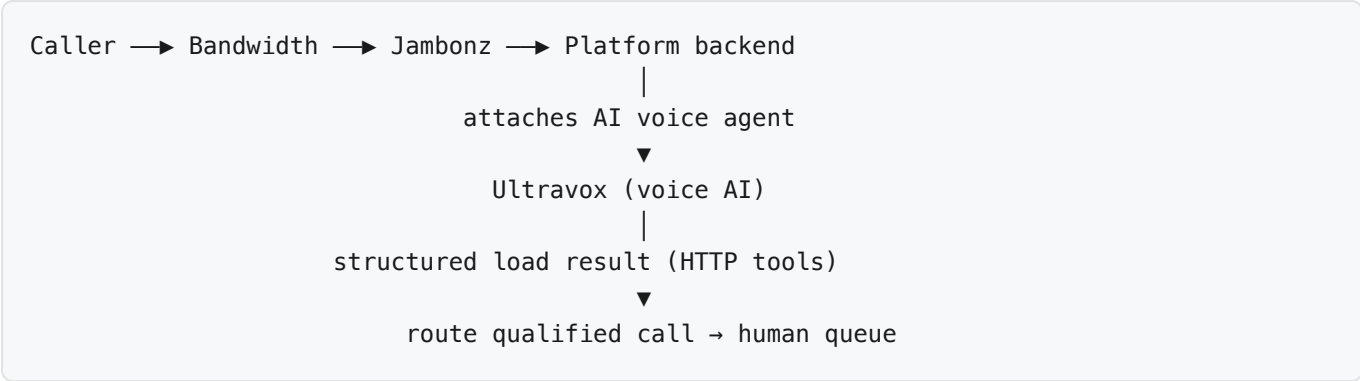
3. Что было поручено ASRP

У клиента уже была реализована **модель звонка** в бэкенде (состояние звонка, «ноги» (legs) звонка, IVR). Мандат ASRP заключался в том, чтобы расширить эту модель и сделать начало звонка интеллектуальным:

1. **Расширить модель звонка**, чтобы бэкенд мог нести **классификацию звонка — КТО / ЗАЧЕМ / ЧТО** — а не только техническую механику звонка.
2. **Сделать IVR интеллектуальным**. На уровне базы данных связать звонящий **номер с конкретным перевозчиком** (чтобы будущий звонок мог проактивно спросить «**ВЫ ЗВОНИТЕ ОТ ЛИЦА X?**»), отслеживать КТО/ЗАЧЕМ/ЧТО между звонками и при **повторном звонке** сразу спрашивать «**тот же перевозчик? тот же груз?**» — давая ту самую преемственность, которой не хватало старому потоку.
3. **Встроить шаг перехода к агенту** в стандартный поток звонка: голосовой агент, которому звонящий может сообщить **что и зачем** он звонит, чей структурированный результат позволяет бэкенду направить звонок нужному человеку.

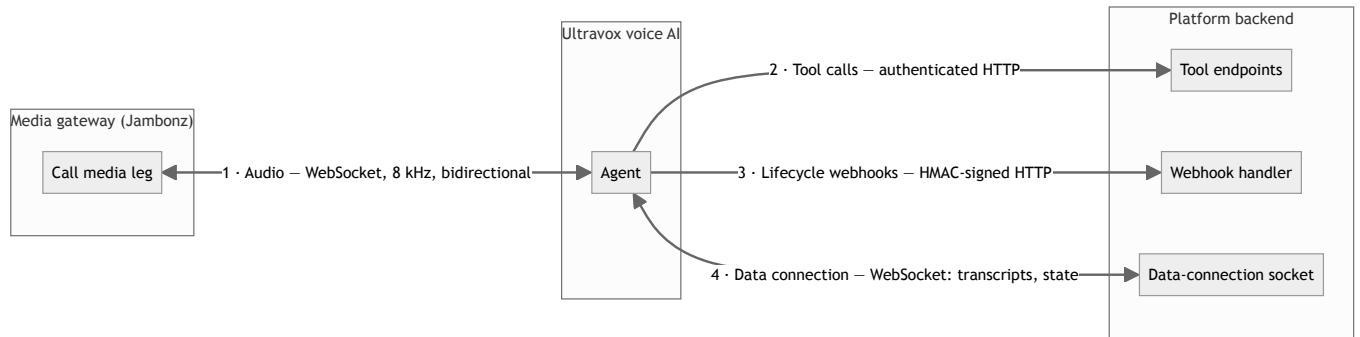
4. Архитектура системы

Хребет звонка — Звонящий → Bandwidth (телефония/SIP) → Jambonz (медиа-шлюз) → бэкенд платформы. Бэкенд подключает ИИ-агента к живому звонку и опосредует каждое взаимодействие с ним.



Интеграция по четырём каналам (ключевое проектное решение)

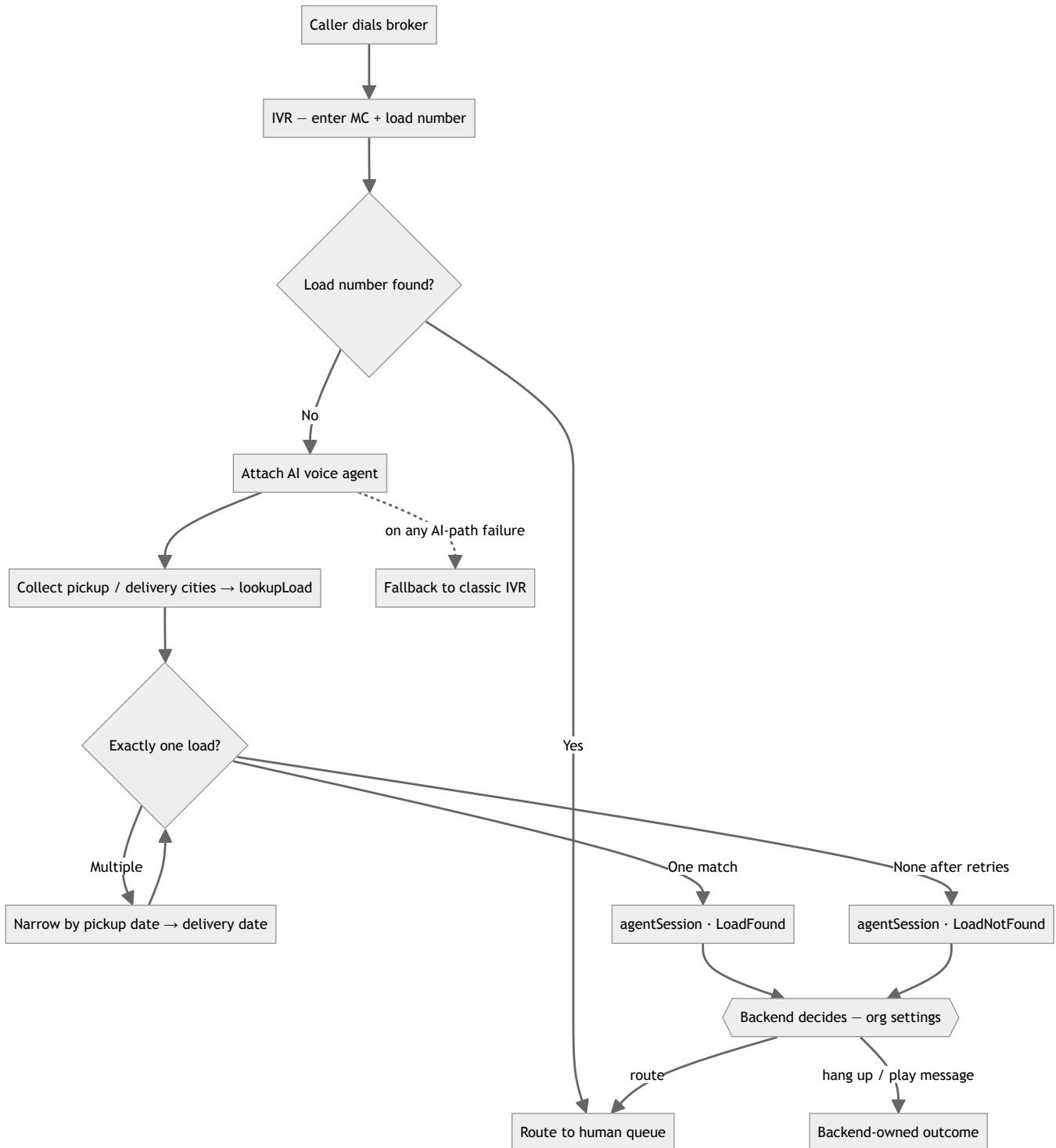
Ключевая идея дизайна в том, что ИИ и бэкэнд общаются по **четырёх независимым каналам**, каждый — отдельный протокол со своей задачей. Чёткое разделение этих каналов — то, что делает интеграцию надёжной.



#	Канал	Протокол	Назначение
1	Аудио	WebSocket, 8 кГц, двунаправленный	Голосовой медиапоток между шлюзом и ИИ
2	Вызовы инструментов (Tool calls)	HTTP POST, аутентификация Bearer	Агент ищет грузы/перевозчиков и сообщает о событиях; всеми данными владеет бэкэнд
3	Webhook'и жизненного цикла	HTTP POST, подпись HMAC-SHA256	Бэкэнд отслеживает состояние звонка (call.started / joined / ended) независимо от диалога
4	Соединение данных (Data connection)	WebSocket	Транскрипты и управляющие сообщения в реальном времени; транскрипты сохраняются и транслируются вживую в интерфейс человека-оператора

5. Пилотный агент

Пилотный агент намеренно **узкий и работает строго «по рельсам»**. Он подключается к звонку **только когда звонящий уже назвал номер груза в IVR, и он не был найден в системе** — то есть агент включается в середине IVR-потока, а не как универсальный чат-бот.



Его единственная задача — **идентифицировать ровно одну отгрузку по голосу**:

- Спросить города/штаты отправления и назначения, затем выполнить поиск.

- Если совпадает несколько грузов, **сузить по дате отправления, затем по дате доставки**; если неоднозначность сохраняется — озвучить варианты и дать звонящему выбрать. Естественные выражения дат («завтра», «в следующую пятницу») нормализуются в формат YYYY-MM-DD .
- Продуктовое наблюдение, заложенное в логику: найти *подходящий* груз, а не обязательно одну точную запись.

Агент работает всего на **двух инструментах** — `lookupLoad` (поиск) и `agentSession` (репортёр событий) — и на двух руководящих принципах, которые делают его надёжным:

- **Сообщай, а не решай.** Агент только *сообщает* о том, что произошло (`LoadFound` , `LoadNotFound` и граничные события); **бэкэнд** решает, переводить звонок или завершать его, основываясь на настройках организации. Агент не имеет права завершить звонок.
- **Голос и инструменты — разные каналы.** При терминальном результате агент вызывает инструмент **молча и перестаёт говорить** — всеми сообщениями, обращёнными к звонящему, владеет бэкэнд. (Небольшая деталь: агент говорит «*Нашёл, секунду*» исключительно для маскировки задержки вызова инструмента.)

Граничные случаи отображаются в заранее определённые события, а не в импровизацию агента: `CallerCannotProvideInfo` , `Silence` , `CallerRequestedHangup` , `AbusiveCaller` и `OffTopicQuestion` (вопросы о тарифах/трекинге/диспетчеризации/оплатах отклоняются один раз, после чего поток продолжается).

6. Как звучит реальный звонок

(Каждое имя, город и референс-номер ниже вымышлены исключительно для иллюстрации — никаких реальных данных о звонящем, перевозчике или грузе.)

Звонящий уже прошёл через IVR и ввёл номер груза, который не совпал — это триггер, описанный в §5. `Bandwidth` и `Jambonz` передают звонок агенту прямо посреди потока; звонящий не подозревает, что трубку только что взяла другая система.

Агент: «Извините, я не смог найти груз с таким номером. Можете назвать города отправления и назначения?»

Звонящий: «Э-э, да — забираю в Модесто, везу в Рино».

(молча: `lookupLoad({ pickupCity: "Modesto", deliveryCity: "Reno" })` — приходит два кандидата)

Агент: «На этом направлении у меня два груза. Знаете дату отправления?»

Звонящий: «Завтра».

(«Завтра» привязывается к фактической дате звонка и преобразуется в YYYY-MM-DD до того, как попадёт в вызов инструмента — никогда не передаётся «как есть». После повторного запроса остаётся один вариант.)

Агент: «Нашёл, секунду».

(Агент запускает вызов инструмента `agentSession(LoadFound)` в тот же момент, когда начинает произносить эту фразу, а не после неё — так звонящий слышит речь, а не тишину, пока бэкэнд фиксирует результат и готовит перевод. После этого агент замолкает: дальше говорит бэкэнд, а не агент.)

Бэкэнд (перевод в очередь): «Соединяю вас».

Второй отрывок показывает, как агент удерживает «рельсы», когда звонящий уходит от темы:

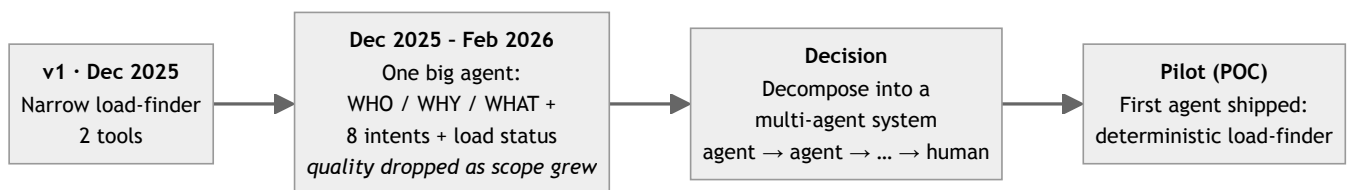
Звонящий: «Слушай, раз уж я на связи — какой тариф на этот груз?»

Агент: «По этой линии я не могу обсуждать тарифы, но давайте сначала разберёмся с грузом — какие города отправления и назначения?»

(`OffTopicQuestion` логируется и отклоняется один раз; поток продолжается. Агент не отказывается от темы навсегда и не уходит в разговор, на который у него нет полномочий.)

7. От одного большого агента к мультиагентной системе

Пилотный «искатель груза» — это один сфокусированный агент, вычлененный из гораздо более богатого агента, который ASRP построила и опробовала в ходе проекта. Эволюция — и ключевое архитектурное решение за ней — выглядела так:



По ходу дела ASRP построила полный набор возможностей первой стадии — изначально внутри одного агента:

- **КТО — идентичность перевозчика.** MC-номер (основной идентификатор), название перевозчика, имя звонящего и его **тип** (`driver` / `dispatcher` / `third_party_dispatch` / `owner_operator`). Перевозчик и звонящий моделируются **раздельно** (как компания и контакт в CRM), потому что один звонящий может представлять несколько перевозчиков. Известные номера подтверждаются («это ABC Freight?»); у неизвестных номеров запрашивается MC.
- **ЗАЧЕМ — классификация намерения.** Таксономия из 8 значений намерения — `LOAD_INQUIRY` (большинство звонков), `CALLBACK`, `POST_BOOKING_SUPPORT`, `ONBOARDING_COMPLIANCE`, `IN_TRANSIT_STATUS`, `BILLING_PAYMENT`, `EXCEPTION_URGENT`, `OTHER_UNKNOWN` — каждое с **оценкой уверенности**; ниже 0.70 агент задаёт один уточняющий вопрос вместо того, чтобы гадать.
- **ЧТО — идентификация груза** с описанной выше воронкой сужения при множественных совпадениях, плюс **обработка статуса груза**: `POSTED` (доступен → продолжить), `UNPOSTED` (недоступен), `DELETED` (уже закрыт) — агент завершает работу только по доступному грузу.
- **Инкрементальные события**, обновляющие состояние звонка в бэкенде по ходу диалога, с терминальным событием завершения, запускающим маршрутизацию.

Почему пилот — это один-единственный «искатель груза». По мере того как зона ответственности этого единого агента росла — классифицировать намерение, идентифицировать перевозчика, найти груз, обработать все граничные случаи — качество выполнения каждой отдельной задачи *ухудшалось*. Более широкий мандат давал более слабого агента. Поэтому вместо того, чтобы заставлять один монолитный агент делать всё, было принято проектное решение **декомпозировать его в мультиагентную систему**: каждый агент владеет узкой задачей и может **передать звонок другому агенту**, когда задача меняется — от агента к агенту, пока звонок не дойдёт до человека. Эта декомпозиция реализована частично (включая разбиение по стадиям, опробованное во фреймворке); продуктовый proof-of-concept **был запущен на первом агенте этой системы — детерминированном «искателе груза»**. (Анализ реальных звонков независимо подтвердил правильность выбора узких агентов: широкий агент оказался хрупким в зашумлённом телефонном канале — например, на неверно распознанных MC-номерах и разговорных датах.)

Защитные механизмы (почему агенту можно доверять)

- Агент **не может завершить звонок** — завершением управляет бэкэнд.
- Агент **не может хранить данные о грузе** в собственной памяти.
- Агент **не может приносить факты**, которые звонящий не называл — каждое сообщаемое значение обосновано поиском в бэкенде (строгая политика данных «использовать только то, что сказал звонящий»).
- Поиск идёт через собственный REST/БД платформы, **а не** через векторное хранилище — результаты детерминированы и проверяемы.

8. Военные истории — самые сложные проблемы в разработке

Четыре момента всплыли в реальном использовании и тестировании, которые разделы архитектуры выше сами по себе не показывают.

1. MC-номера не переживали телефонную линию. Произнесённые голосом цифровые последовательности — особенно MC-номера — это ровно тот тип входных данных, который телефонный аудиоканал искажает: пропущенные цифры, переставленные цифры, помехи. Анализ реальных звонков (зафиксированный тестовым фреймворком из §13) выявил это как повторяющийся сбой и напрямую повлиял на решение из §7 держать поставленного в продакшн агента узким, а не бросать широкий классификатор на зашумлённый канал: широкому агенту, которому нужно было правильно получить MC-номер, и намерение, и груз за один проход, было некуда «поглотить» эту ошибку. Смягчение, заложенное в промпт — порог уверенности: ниже 0.70 агент задаёт уточняющий вопрос вместо того, чтобы гадать, — а не какая-либо попытка коррекции на уровне аудио.

2. Естественные даты не разбирались так, как их произносили звонящие. «Завтра», «в следующую пятницу», «послезавтра» — обманчиво просто для человека, неоднозначно для модели, если не заякорено. Ранее поведение иногда передавало слова звонящего напрямую в вызов инструмента или разрешало относительную дату относительно неверной точки отсчёта. Решение: промпт несёт явный якорь `{{ todayDate }}` плюс разобранную таблицу преобразований (`"tomorrow"` → сегодня + 1 день → `YYYY-MM-DD`) и парные примеры «правильно/неправильно», так что нормализация дат происходит как жёсткое правило до того, как значение увидит хоть один инструмент — а не как вывод, который агент делает на лету.

3. Агент выдумывал детали, которых звонящий не называл. Самым явным риском галлюцинации была не выдумка данных о грузе — это структурно исключено защитными механизмами из §7 — а нечто более мелкое: агент иногда сам заполнял штат назначения, который звонящий фактически не упоминал, выводя его из контекста или из голого пункта отправления. Решение — прямолинейная политика данных, закреплённая в промпте: использовать *только* те поля, которые звонящий явно назвал, поле за полем — «груз из Нью-Мексико» заполняет только `originState` и ничего больше, даже если пункт назначения кажется очевидным.

4. Задержка вызова инструмента создавала «мёртвый эфир» — решено разговором во время ожидания, а не молчаливым ожиданием. Каждый вызов `lookupLoad / agentSession` — это сетевой round-trip, и молчаливый разрыв в несколько сотен миллисекунд на телефонном звонке воспринимается как обрыв или потеря связи. Вместо того чтобы ждать результата инструмента перед тем, как заговорить, агенту предписано произносить короткую связующую фразу («Нашёл, секунду») **одновременно** с запуском вызова инструмента — голос и вызов инструмента идут параллельно, а не последовательно. Если перепутать порядок — вызвать инструмент молча и заговорить только после — это прямо обозначено как неверное поведение в собственных разобранных примерах промпта: тишина во время вызова инструмента воспринимается звонящим как задержка, даже если сам вызов на самом деле быстрый.

9. Технологический стек

Слой	Технология
Диалоговый голосовой ИИ	Ultravox (речевая нативная модель, STT+LLM слиты воедино; модель <code>ultravox-v0.7</code>)
Синтез речи (Text-to-speech)	InWorld , голос «Sarah» — подобран в соответствии с голосом IVR платформы на Jambonz для бесшовной передачи
Программируемая голосовая связь / медиа	Jambonz
Телефония / SIP	Bandwidth
Бэкенд	TypeScript, Node.js с веб-фреймворком Hono
Транспорт реального времени	WebSockets (аудио + соединение данных)
Ретрансляция событий в UI	NATS pub/sub → сервер WebSocket → фронтенд
Персистентность	MySQL через Drizzle ORM — PlanetScale serverless в продакшне, mysql2 (MySQL 9) для локальной разработки; кэширование/очереди на Redis
Тестовый контур	Отдельный проект на Bun/TypeScript (Puppeteer для полноценных голосовых тестов через браузер)

О модели ИИ: Ultravox использует собственную открытую модель на базе LLaMA, которая понимает голосовые токены напрямую (распознавание речи и рассуждение слиты воедино, что снижает задержку). Выбор базовой LLM недоступен пользователю на платформе Ultravox — это принятое ограничение платформы, компенсированное строгой детерминированностью промпта/инструментов и тем, что данными владеет бэкенд.

10. Модель данных

Интеграция опирается на реляционную SQL-схему (Drizzle ORM). Три таблицы центральны:

- `active_calls` — центральная строка состояния звонка, привязанная к сессии Ultravox через индексированный корреляционный ключ `aiSessionId`. Содержит контекст организации/маршрутизации, «ноги» звонка, поля перевозчика/звонящего/груза, заполняемые потоком вызовов инструментов агента (название компании, номер груза, номера MC/DOT), человекочитаемый `currentStatus` и колонки COMPLAINTS/метаданных. Собранные данные также сериализуются в JSON в записи звонка.
- `ultravox_webhooks` — журнал событий жизненного цикла типа append-only (`ultravoxCallId`, `webhookEvent`, полный JSON-payload), записываемый **идемпотентно**, индексированный по `id` звонка / событию / времени.
- `ultravox_transcripts` — хранилище транскриптов в реальном времени: `role` (пользователь/агент), `medium` (голос/текст), `text`, `ordinal`, флаг `final`, ключ — `id` звонка. Идемпотентность обеспечивается по паре (`ultravoxCallId`, `ordinal`).

«Какую базу данных вы использовали?» → **MySQL**, доступ через Drizzle ORM на TypeScript из общего пакета схемы. Платформа работает по стратегии двойного подключения: **PlanetScale serverless** (MySQL) в продакшне и драйвер `mysql2` (MySQL 9) для локальной разработки. ASRP добавила указанные выше таблицы для фичи голосового ИИ и расширила существующую таблицу состояния звонка.

11. API и поверхность интеграции

Инструменты агента (ИИ → бэкэнд, аутентифицированный HTTP POST):

Инструмент	Статус	Назначение
<code>lookupLoad</code>	пилот	Поиск груза по референс-номеру / городам / датам; возвращает грузы-кандидаты со статусом (posted/unposted/deleted)
<code>agentSession</code>	пилот	Сообщает о классификации и событиях жизненного цикла; терминальное событие запускает маршрутизацию в бэкэнде
<code>lookupEntity</code>	построен (более широкая конфигурация)	Поиск перевозчика по MC или DOT номеру; обновляет поля перевозчика в записи звонка

Webhook’и жизненного цикла (ИИ → бэкэнд): `call.started`, `call.joined`, `call.ended`. Эндпоинт подтверждает получение немедленно и обрабатывает событие в фоне, записывая каждое событие идемпотентно. Обработчик `call.ended` сверяет финальное состояние и гарантирует, что звонок попадёт в очередь даже на путях отката (fallback).

Безопасность: webhook'и верифицируются через **HMAC-SHA256** над `timestamp + "." + body`, с окном свежести подписанной метки времени и **поддержкой ротации ключей** (во время ротации принимается несколько валидных подписей). Вызовы инструментов аутентифицируются по Bearer-токену.

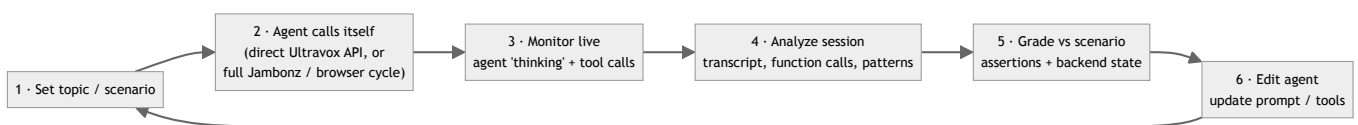
12. Надёжность и эксплуатация

- **Полностью управляется фича-флагом** — единый флаг переключает весь ИИ-путь целиком; когда он выключен, платформа прозрачно использует классическое IVR.
- **Автоматический откат (fallback)** — настраивается на уровне телефонного номера; при любом сбое на ИИ-пути звонок автоматически откатывается к жёсткому IVR.
- **Идемпотентная запись** — сохранение webhook'ов и транскриптов идемпотентно, поэтому повторные/дублирующие доставки никогда не портят состояние.
- **Передача с низкой задержкой** — быстрый путь постановки в очередь запускает передачу человеку в момент идентификации груза, минимизируя «мёртвый эфир».

13. Контур разработки ИИ (фреймворк самообучения)

Итерировать голосового агента вручную медленно: позвонить, поговорить с ним, догадаться, что пошло не так, отредактировать промпт, позвонить снова. ASRP построила отдельный **фреймворк на Bun/TypeScript**, который замыкает этот контур и позволяет ИИ прямо в IDE **развивать агента самостоятельно** — небольшую мета-обучающуюся систему, где агент программно редактируется, вызывается и оценивается. Это **отдельный репозиторий** от продакшн-платформы (нулевая зависимость от продакшна; фреймворк *расширяет* общий пакет Ultravox утилитами обновления агента и работы с промптом/инструментами), поэтому этот инструментарий никогда не попадает в продакшн-код.

Контур полностью замкнут — ИИ может отредактировать агента, вызвать его, оценить звонок и применить исправление, а затем повторить:



- **Агент звонит и разговаривает сам с собой.** Прогон задаёт тему разговора или заранее заданный тест-кейс, затем агент совершает звонок — либо **напрямую в Ultravox** (в обход PBX; быстро, для отладки поведения и вызовов инструментов), либо через **полный конвейер Jambonz / браузер** (создавая настоящую запись звонка и webhook'и для сквозной проверки). Режим прямого WebSocket-подключения и режим на основе Puppeteer для полноценного голоса дополняют набор методов.
- **Он наблюдает за «мышлением» агента.** Сессии отслеживаются в реальном времени — история чата, вызовы инструментов/функций и паттерны их использования — так что сбои диагностируются по реальным рассуждениям агента, а не угадываются.
- **Он оценивает по сценариям.** Именованные сценарии прописывают реплики звонящего и несут явные **утверждения (assertions)**: ожидаемая классификация намерения, извлечённые поля (номер груза, города, MC), инструменты, которые должны сработать, «агент замолчал

после терминального события» и — для полноцикловых прогонов — результирующее состояние бэкенда (звонок поставлен в очередь, записи записаны). Тест-кейсы охватывают всю таксономию намерений.

- **Он редактирует агента прямо из проекта.** Промпты и инструменты агента обновляются программно из фреймворка, а этап анализа может предложить изменения для применения — так что цикл диагностика → исправление → повторный тест проходит без выхода из кодовой базы.
- **Корпус сессий.** Каждый прогон записывается (полный транскрипт, события и полный системный промпт, действовавший в момент сессии) для офлайн-анализа и регрессионного разбора. Именно этот корпус выявил проблемы с цифрами МС и разбором дат, описанные в §8.

14. Целевые показатели проектирования

Интеграция проектировалась с прицелом на следующие цели по голосовому UX и точности (см. §15 об итогах пилота):

Показатель	Цель
Задержка классификации намерения	< 2 с
Первый ответ агента	< 3 с
Вся фаза квалификации	< 30 с
Задержка вызова инструмента	< 1 с
Точность классификации намерения	> 90%
Доля успешного захвата данных звонящего/груза	> 85%
Точность маршрутизации	> 95%
Аптайм	> 99.5%

15. Результаты и итоги

Proof-of-concept сработал. В живом тестировании агент обрабатывал реальные входящие звонки от начала до конца — вёл диалог со звонящими, идентифицировал грузы по голосу и передавал квалифицированные звонки в очередь на человека.

Система была поставлена **готовой к продакшну, но пока не включена для реального клиентского трафика**. Включение — это решение о выходе на рынок (go-to-market), которое остаётся за клиентом, и оно не было принято к моменту завершения проекта — сборка полностью завершена и готова к активации в любой момент, когда клиент решится. Оставшийся шаг — бизнес-решение, а не дальнейшая инженерная работа.

16. Зрелость и дорожная карта

Пилотный охват (Фаза 1 — детерминированная): когда звонящий не может назвать валидный номер груза через IVR, звонок переключается на ИИ-агента, который идентифицирует груз по голосу (по референс-номеру либо по пункту отправления/назначения и датам) и ставит звонок в очередь на человека. Поведение намеренно узкое и «на рельсах»: минимум вопросов, никакого разговора не по теме.

Частично построено — мультиагентная декомпозиция: целевая архитектура — это набор узкоспециализированных агентов, передающих звонок друг другу (пока он не дойдёт до человека), вместо одного монолитного агента, качество которого падает по мере роста его зоны ответственности (см. §7). «Искатель груза» — первый агент; фреймворк передачи звонка построен частично.

Спроектировано, но пока не поставлено:

- **Движок принятия решений о маршрутизации на бэкенде** — ACCEPT / REJECT / HOLD на основе статуса комплаенса, состояния блокировки перевозчика, доступности груза, настроек организации и анти-спама (например, ограничение частоты повторных звонков). Сообщения об отказе генерирует бэкэнд, а не агент.
- **Интеллект по ANI (caller ID)** — кэш поиска по номеру звонящего и выделенные таблицы классификации намерений, чтобы известный номер распознавался мгновенно, а предыдущие звонки информировали текущий диалог.

Будущие фазы:

- **Фаза 2 (диалоговая):** переговоры, ставки, обновления статуса, когда КТО/ЗАЧЕМ/ЧТО уже известны.
- **Фаза 3 (исходящая):** агент сам инициирует звонки.

17. Что поставила ASRP

ASRP полностью владела **всем воркстримом голосового ИИ** поверх существующей платформы звонков клиента:

- Разработала **стратегическую дорожную карту** для двухфазной (детерминированная → диалоговая) архитектуры голосового ИИ и модели КТО/ЗАЧЕМ/ЧТО.
- Спроектировала и реализовала все четыре канала коммуникации ИИ↔бэкэнд (подключение аудио, HTTP-инструменты, подписанные webhook'и, соединение данных в реальном времени).
- Расширила модель звонка бэкенда для переноса классификации звонка и построила интерфейс инструментов агента (`lookupLoad` , `agentSession` , `lookupEntity`).
- Построила и итеративно дорабатывала поведение агента — от полного 8-интентного классификатора до детерминированного пилотного «искателя груза» — написав системный промпт и конфигурацию инструментов на основе записанных сессий.
- Реализовала захват транскриптов в реальном времени с идемпотентной персистентностью и трансляцией вживую в интерфейс человека-оператора.
- Построила отдельный фреймворк тестирования / самообучения.

18. Часто задаваемые вопросы (для разговоров с клиентами)

В: Что реально работает в пилоте? Узкий, детерминированный агент: он подключается только тогда, когда номер груза, названный звонящим, не найден, идентифицирует отгрузку по голосу и сообщает результат, чтобы бэкенд мог направить звонок. Более полный классификатор намерений был построен и доработан, но придержан за этим надёжным подмножеством функциональности.

В: Какую базу данных вы использовали? MySQL через Drizzle ORM на TypeScript — PlanetScale serverless в продакшне, драйвер mysql2 (MySQL 9) локально. Выделенные таблицы для состояния звонка, логирования webhooks и транскриптов, Redis для кэширования/очередей.

В: Какая LLM управляет агентом? Собственная речевая нативная модель Ultravox (на базе LLaMA, понимающая голосовые токены). Платформа не раскрывает выбор LLM; мы компенсировали это строгой детерминированностью промпта, фиксированным интерфейсом инструментов и тем, что данными владеет бэкенд, — так что модель ведёт диалог, но никогда не выдумывает.

В: Как вы не даёте ему галлюцинировать данные об отгрузке? Жёсткие защитные механизмы: агент может сообщать только значения, возвращённые поисками бэкенда, не может выдумывать детали и не может хранить данные в собственной памяти. Поиск детерминирован (REST/БД бэкенда, а не векторное хранилище).

В: Что происходит, если ИИ сбоит посреди звонка? Автоматический откат к классическому IVR, настраиваемый по номеру. Вся фича управляется единым флагом, поэтому её можно отключить мгновенно без деплоя.

В: Это безопасно? Webhook'и подписаны HMAC-SHA256 с окном свежести и ротацией ключей; вызовы инструментов аутентифицированы по Bearer-токену; все записи событий/транскриптов идиempotentны.

В: Как это тестируется? Выделенный фреймворк с четырьмя методами тестирования (от прямого сокета до полноценного голосового теста через браузер), сценарии со скриптованными утверждениями и записанный корпус сессий для регрессионного анализа.

Обезличенное описание проекта подготовлено ASRP для портфолио. Название клиента, пути к проприетарному исходному коду, внутренние идентификаторы и детали продакшн-инфраструктуры намеренно опущены из текста. Более глубокие технические подробности доступны по NDA.

1. **Ultravox** — речевая нативная голосовая ИИ-платформа, модель которой потребляет аудио напрямую (распознавание речи и рассуждение слиты воедино, а не проходят через отдельный этап транскрипции), что снижает задержку диалога. <https://www.ultravox.ai>↵
2. **Jambonz** — платформа программируемой голосовой связи / CPaaS с открытым исходным кодом, соединяющая телефонию (SIP/PSTN-операторов) с приложениями и голосовыми ботами, выступающая медиа-шлюзом между телефонной сетью и ИИ. <https://jambonz.org>↵

3. **Bandwidth** — американский оператор связи / CPaaS-провайдер, предоставляющий SIP-транкинг и подключение к PSTN (номера и голос) — телефонный слой, который несёт сам звонок. <https://www.bandwidth.com>↩